CS 432: Databases

Assignment 03
Module A

Track 01

by
Nitin Bansal
Devvrat Hans
Sailaja Mudu
Thipparapu Rushitha
Vadithya Harsha Vardhan Nayak
of Group 21

Bachelor of Technology,
Indian Institute of Technology Gandhinagar

Contents

Abstract ……...…….……3

Introduction ……...……..……3

System Architecture ……...…..4

Transaction Workflow and ACID Enforcement…….…...7

1. **ACID Enforcement**……...…..8
2. **Multi-Table Workflow Demonstration**……..…9

How Correctness of Operations is Ensured….……..9

1. **Structural Consistency Validation**……9
2. **Safe Commit Promotion**……..……...10
3. **CRUD Correctness Validation** ……10

How Failures are Handled

1. **Failure Injection During Runtime**……..……..11
2. **Write-Ahead Logging and Rollback Metadata**……...……..11
3. **Rollback on Injected Failure**……12
4. **Restart and Crash Recovery**……....13
5. **Experimental Evidence**……..……..…13

Experiments Performed and Analysis……...…13

Limitations and Future Improvements……...…17

Conclusion…….....18

Team Contribution Table……...…..18

Github Repository Link

- *https://github.com/devvrat-hans/CS-432-2026.git*

Video Link:

- *https://drive.google.com/drive/folders/1Fz0fKlavOVdpYByTO0uZleCLBMdxQGqN?usp=sharing*

Module A: Advanced Transaction Engine & Crash Recovery using B+ Tree (ACID Validation)

Abstract

This report presents the implementation of Module A: Transaction Management, Crash Recovery, and ACID Validation for a custom database system built over a B+ Tree indexing structure. The primary objective of this module is to ensure that database operations remain correct, reliable, and recoverable under both normal execution and failure scenarios.

The implemented system supports transactional execution of insert, update, and delete operations, where every operation either completes fully or is safely rolled back in case of failure. To achieve this, a logging-based recovery mechanism is integrated into the database manager, enabling rollback of incomplete operations and restoration of committed data after restart. The design also guarantees strict consistency between the persistent database records and the B+ Tree index, ensuring that search and retrieval operations always reflect the correct database state.

To validate correctness, multiple experiments were conducted including failure injection during transactions, rollback verification, restart recovery, and B+ Tree consistency checks. The results confirm that the system satisfies the key ACID properties of Atomicity, Consistency, and Durability, while maintaining robustness under crash scenarios. This implementation establishes a reliable transaction engine that forms the foundation for safe and scalable database operations.

Introduction

In modern database systems, ensuring the correctness and reliability of data is critical, especially in the presence of system failures and concurrent operations. This module focuses on enhancing a previously developed B+ Tree-based database system by introducing transaction management, crash recovery, and ACID guarantees.

The primary goal of this module is to ensure that the system behaves correctly under both normal execution and failure conditions. Unlike traditional database architectures, where storage and indexing are separate, this system uses the B+ Tree as the sole storage and indexing structure. All records are directly stored and managed within the B+ Tree, ensuring strong consistency between the database and its index.

To achieve reliability, a transaction framework supporting BEGIN, COMMIT, and ROLLBACK operations is implemented. Each transaction is executed using a staged mutation approach, where changes are first applied to a temporary copy of the database and committed only after validation. This prevents partial updates and guarantees safe execution.

Additionally, a Write-Ahead Logging (WAL) mechanism is introduced to record all changes before they are applied. This enables the system to recover from crashes by either reapplying committed transactions

or undoing incomplete ones. A robust rollback mechanism ensures that any failed transaction restores the database to its previous consistent state.

The system is designed to satisfy the four fundamental ACID properties:

- Atomicity, ensuring transactions are fully completed or fully rolled back
- Consistency, maintaining valid database states after each transaction
- Isolation, preventing interference between concurrent transactions
- Durability, ensuring committed data persists across system restarts

Through this module, the database system is transformed into a reliable and fault-tolerant engine capable of handling failures while maintaining correctness and integrity of data.

2. System Architecture

The implemented transaction engine is built as a layered architecture over the custom B+ Tree–based database system. The design separates persistent storage, transaction processing, logging, recovery, and validation into clearly defined components so that correctness can be maintained across normal execution as well as failure scenarios.

At a high level, the system consists of the following major components:

2.1 Persistent Storage Layer

The persistent storage layer is responsible for maintaining the actual database records on disk. This functionality is implemented inside the database/ module, which serves as the main storage backend of the system.

This layer manages:

- creation of tables
- persistent record storage
- update and deletion of records
- table-level metadata
- recovery of stored data after restart

All committed data modifications are ultimately reflected in this layer, making it the source of truth for the database state.

2.2 B+ Tree Index Layer

To enable efficient searching and ordered access, the database uses a custom B+ Tree index structure. The B+ Tree maintains indexed references to database records and supports:

- insertion of new keys

- deletion of keys
- search operations
- range-based queries
- tree visualization

A critical design requirement of Module A is that the B+ Tree must always remain synchronised with the persistent storage layer, even when failures occur midway through execution.

2.3 Transaction Management Layer

The transaction management layer forms the core of Module A. It ensures that every database operation follows transactional semantics.

This layer is demonstrated using:

- demo_transaction.py
- btree_acid_advanced_demo.py

The transaction workflow includes:

1. Begin Transaction
2. Record operation in the log
3. Apply a modification to the database
4. Update B+ Tree index
5. Commit changes

If any failure occurs before commit, the transaction manager triggers a rollback and restores the previous stable state.

This layer ensures:

- complete execution of valid operations
- rollback of partial updates
- prevention of inconsistent intermediate states

2.4 Logging and Recovery Layer

The logs/ directory serves as the engine's write-ahead logging subsystem. Every critical update is first recorded in the log before modifying the actual database or index.

This layer supports:

- rollback of incomplete operations
- replay of committed operations after restart
- crash recovery validation

- failure diagnosis

By storing sufficient before-state and after-state information, the system can recover safely from simulated crashes.

The recovery mechanism is extensively validated using:

- test_restart_recovery.py
- test_full_system_validation.py

2.5 Validation and Testing Layer

To verify the correctness of the architecture, multiple dedicated testing scripts are included.

Core validation files

- test_acid_validation.py → validates Atomicity, Consistency, Durability
- test_bplustree_acid.py → validates DB and B+ Tree consistency
- test_restart_recovery.py → validates durability after restart
- test_reliability_edge_cases.py → validates abnormal failure cases
- test_hardening.py → validates resilience under unexpected conditions
- test_full_system_validation.py → integrated end-to-end correctness validation

These scripts provide strong evidence that the transaction engine behaves correctly across all critical scenarios required in Module A.

2.6 Architectural Workflow

The overall architecture follows the workflow shown below:

Client Operation
↓
Begin Transaction
↓
Write-Ahead Log Entry
↓
Update Database Record
↓
Update B+ Tree Index
↓
Commit
↓
Persist Success

Failure path

Client Operation
↓
Begin Transaction
↓
Partial Update
↓
System Failure
↓
Read Logs
↓
Rollback / Recovery
↓
Restore DB + B+ Tree

This workflow ensures that no partial or invalid state survives a failure.

3. Transaction Workflow and ACID Enforcement

The implemented database engine follows a transaction-oriented staged mutation workflow designed to guarantee correctness, rollback safety, and recovery under failure scenarios. The transaction lifecycle begins with the *begin()* , which internally creates and activates a TransactionContext. A BEGIN record is immediately written to the Write-Ahead Log (WAL) to ensure recoverability from the earliest point of execution

Each data-modifying operation, such as create_table, insert, update, or delete, is executed through the staged mutation engine *_run_staged_mutation()*. Instead of directly modifying the live database state, the engine first creates:

- a baseline snapshot of the current tables
- a staged snapshot for isolated modifications

The requested mutation is then applied only to the staged copy. This ensures that partially completed operations never affect the committed state of the database.

Once the staged mutation completes, the engine performs:

1. index-write checkpoint validation
2. WAL staging log generation
3. constraint validation
4. database consistency verification
5. commit marker logging

Only after all these checks succeed is the staged snapshot promoted to the live database state. This enforces an atomic transition from the old state to the new state.

If any failure occurs at predefined checkpoints, such as:

- after index write
- after WAL write
- after data write
- before commit marker

The engine restores the baseline snapshot and writes explicit rollback WAL entries. This guarantees that the database always returns to the exact pre-transaction state.

3.1 ACID Enforcement

The workflow enforces the key ACID properties as follows:

Atomicity

Each mutation is applied to staged snapshots and either:

- fully replaces the committed state
- fully restores the baseline snapshot on failure

This was experimentally validated using injected failures during inserts and updates.

Consistency

After every staged mutation, the system executes:

- duplicate key validation
- balance and stock constraints
- sorted B+ Tree verification
- record count consistency checks

This ensures that all table states remain valid after commit.

Durability

All staged and committed images are persisted in the WAL using before-images and after-images. During restart, *recover_from_wal()* reconstructs the committed state by replaying committed transactions and undoing incomplete ones. Multiple restart experiments confirmed persistence correctness.

Isolation

The current implementation follows an operation-level auto-commit isolation model, where each mutation is independently staged and committed. Uncommitted partial writes are not exposed because the live state is updated only after consistency validation. This model provides strong single-operation isolation and serves as the base for future lock-based concurrency control.

3.2 Multi-Table Workflow Demonstration

The transaction workflow was demonstrated using a simulated order-processing use case involving:

- users
- products
- orders

The workflow performs:

1. user balance deduction
2. stock decrement
3. order insertion

In the current auto-commit model, each operation is individually staged and committed, while still preserving correctness and table-level consistency. Final consistency reports confirmed:

- no duplicates
- sorted index order
- matching record counts across tables.

4. How Correctness of Operations is Ensured

Correctness of operations in the implemented database engine is ensured through a combination of constraint validation, staged snapshot updates, structural consistency verification, and CRUD correctness testing. Every mutating operation first executes on a staged copy of the database state and is subjected to multiple correctness checks before becoming visible as a committed state.

The first level of correctness enforcement is implemented through the constraint validation engine using *_validate_constraints()*. Before staged data is promoted to the live database, the system scans all affected records and validates application-level invariants. In the current implementation, two critical semantic constraints are enforced:

- balance must never become negative
- stock count must never become negative

If either condition is violated, the transaction is aborted immediately, and a rollback path is triggered. This prevents logically invalid states, such as overdraft balances or negative product inventory, from ever being committed.

4.1 Structural Consistency Validation

After semantic validation, the system performs a database-wide structural consistency check using *_assert_database_consistency()*. This method collects consistency reports from all active tables and verifies that:

- no duplicate keys exist
- record ordering remains sorted
- B+ Tree index size matches actual record count
- table-level invariants remain valid

If any inconsistency is detected, the engine raises a runtime exception with detailed diagnostics, preventing invalid structural states from entering the committed database.

This guarantees that correctness is preserved not only at the logical record level but also at the index and table structure level.

4.2 Safe Commit Promotion

The correctness pipeline during staged mutation follows the sequence:

```
Constraint Validation
        ↓
Failure Checkpoint Validation
        ↓
Promote Staged Snapshot
        ↓
Database Consistency Assertion
        ↓
Commit
```

This design ensures that the live database state is updated only after all correctness checks succeed.

4.3 CRUD Correctness Validation

The correctness of standard database operations was further validated using the DB Manager Demo.

The experimental output confirmed correct behaviour for:

- insert
- update
- delete
- range query

Observed output:

```
=== DB MANAGER DEMO ===
After insert: [(1, 'Alice'), (2, 'Bob')]
After update: [(1, 'Alicia'), (2, 'Bob')]
After delete: [(1, 'Alicia')]
Range query: [(1, 'Alicia')]
 DEMO COMPLETED
```

These results verify that the database manager correctly preserves record integrity across the full CRUD lifecycle while maintaining index correctness.

The successful CRUD demo and ACID validation logs further confirm that the engine consistently maintains valid state transitions across normal and failure-driven execution paths.

5. How Failures are Handled

The implemented transaction engine uses a multi-layer failure handling framework based on failure injection checkpoints, Write-Ahead Logging (WAL), rollback metadata generation, and crash recovery replay. The primary objective of this design is to ensure that the database never persists incomplete or corrupted state, even if failures occur during any phase of execution.

5.1 Failure Injection During Runtime

To validate rollback safety under abnormal execution paths, the system includes an explicit failure injection mechanism through configure*_failure_injection()*. This module allows deterministic crashes to be triggered at critical checkpoints of the transaction lifecycle:

- after_index_write
- after_log_write
- after_data_write
- before_commit_marker

The method _check_failure_checkpoint() decrements the configured trigger counter and raises a runtime exception once the specified checkpoint is reached. This allows precise simulation of failures during index updates, WAL persistence, data writes, and commit marker generation.

This design was used extensively to verify that partial updates never survive injected crashes.

5.2 Write-Ahead Logging and Rollback Metadata

Before any transaction is considered durable, the system writes detailed before-image and after-image WAL events using *_build_wal_events()*.

Each WAL record stores:

- transaction identifier
- timestamp
- target entity
- record key
- operation type
- before image
- after image
- transaction status

This ensures that enough information is always available to either:

- replay committed transactions
- restore previous state after rollback

In failure scenarios, *_build_rollback_wal_events()* generates explicit rollback markers where:

- before_image = original stable state
- after_image = restored stable state
- status = rolled_back

This makes rollback operations fully traceable and durable in the WAL log.

5.3 Rollback on Injected Failure

When a failure is triggered during staged mutation, the engine restores the baseline snapshot of the database and records rollback events in WAL.

A validated runtime example from the WAL output shows:

status: "staged"

before_image: {"name": "Alice", "balance": 1000}

after_image: {"name": "Alice", "balance": 500}

followed by:

status: "rolled_back"

before_image: {"name": "Alice", "balance": 1000}

after_image: {"name": "Alice", "balance": 1000}

error_message: "Injected failure at checkpoint 'after_data_write'"

This confirms that the rollback engine successfully restored the exact pre-transaction balance after the simulated failure.

5.4 Restart and Crash Recovery

The restart recovery engine is implemented in recover_from_wal().

During system startup, the WAL is scanned and grouped by transaction identifiers.

The recovery logic applies:

- REDO for committed transactions
- UNDO for incomplete transactions

If a transaction contains a committed status, the engine restores the after_image.

Otherwise, staged or failed transactions are processed in reverse order using their before_image.

This guarantees that:

- committed updates always survive restart
- incomplete transactions are fully undone

- malformed WAL entries are safely ignored
- recovery statistics are preserved for diagnostics

This design was experimentally validated through repeated restart simulations and forced crash recovery tests.

5.5 Experimental Evidence

The WAL log output clearly demonstrates successful handling of:

- table creation failures
- record update rollback
- after-data-write crash injection
- restart recovery replay
- rollback restoration of previous balance and stock states

These experiments confirm that the database remains reliable under both runtime failures and restart-level crashes.

6. Experiments Performed and Analysis

To systematically validate the correctness and robustness of the implemented transaction engine, we designed a multi-layer experimental methodology consisting of 17 automated tests, B+ Tree ACID validation, and CRUD workflow demonstrations.

The experiments were not limited to simple unit checks. Instead, they were structured to verify the system across:

- transaction correctness
- failure injection checkpoints
- rollback restoration
- restart recovery
- WAL redo/undo
- duplicate prevention
- constraint safety
- B+ Tree index integrity
- CRUD functional correctness

```
PS C:\Users\saila\OneDrive\Desktop\assignment_3_mA (3)\assignment_3_mA> python -m unittest -v | tee final_demo_output.txt
test_atomicity_failure_rolls_back (test_acid_validation.TestACIDValidation.test_atomicity_failure_rolls_back) ... ok
test_durability_after_restart (test_acid_validation.TestACIDValidation.test_durability_after_restart) ... ok
test_multi_table_transaction (test_acid_validation.TestACIDValidation.test_multi_table_transaction) ... ok
test_rollback_restores_previous_state (test_acid_validation.TestACIDValidation.test_rollback_restores_previous_state) ... ok
test_atomicity_failure_rolls_back (test_full_system_validation.TestFullSystemValidation.test_atomicity_failure_rolls_back) ... ok
test_consistency_constraint_violation (test_full_system_validation.TestFullSystemValidation.test_consistency_constraint_violation) ..
. ok
test_durability_after_restart (test_full_system_validation.TestFullSystemValidation.test_durability_after_restart) ... ok
test_failure_after_log_write (test_full_system_validation.TestFullSystemValidation.test_failure_after_log_write) ... ok
test_failure_before_commit (test_full_system_validation.TestFullSystemValidation.test_failure_before_commit) ... ok
test_isolation_uncommitted_not_visible (test_full_system_validation.TestFullSystemValidation.test_isolation_uncommitted_not_visible)
... ok
test_recovery_redo_and_undo (test_full_system_validation.TestFullSystemValidation.test_recovery_redo_and_undo) ... ok
test_wal_before_commit_crash (test_full_system_validation.TestFullSystemValidation.test_wal_before_commit_crash) ... ok
test_duplicate_key_no_partial_state (test_hardening.TestHardening.test_duplicate_key_no_partial_state) ... ok
test_failure_rollback (test_hardening.TestHardening.test_failure_rollback) ... ok
test_recovery_after_operations (test_reliability_edge_cases.TestReliabilityEdgeCases.test_recovery_after_operations) ... ok
test_recovery_after_operations (test_restart_recovery.TestRestartEdgeCases.test_recovery_after_operations) ... ok
test_recovery_with_failure (test_restart_recovery.TestRestartEdgeCases.test_recovery_with_failure) ... ok

----------------------------------------------------------------------
Ran 17 tests in 0.415s

OK
```

```
=== B+ TREE ACID VALIDATION ===

[Atomicity Test]
After rollback: [(1, 'A'), (2, 'B')]
 Atomicity verified (no partial inserts)

[Consistency Test]
Tree data: [(1, 'A'), (2, 'B'), (10, 'X'), (20, 'Y')]
Consistency: [{'table': 'btree_test', 'ok': True, 'checks': {'no_duplicates': True, 'is_sorted': True, 'size_matches': True}, 'record
_count': 4}]

[Durability Test]
After restart: [(1, 'A'), (2, 'B'), (10, 'X'), (20, 'Y'), (99, 'Persist')]

[Isolation Test]
After insert: Temp
 Isolation model: auto-commit

=== B+ TREE ACID TEST COMPLETED ===

=== DB MANAGER DEMO ===
After insert: [(1, 'Alice'), (2, 'Bob')]
After update: [(1, 'Alicia'), (2, 'Bob')]
After delete: [(1, 'Alicia')]
Range query: [(1, 'Alicia')]
 DEMO COMPLETED
```

6.1 ACID Validation Test Suite

The first experiment group focused on the core ACID guarantees of the database engine.

Test cases performed

1. test_atomicity_failure_rolls_back
2. test_durability_after_restart
3. test_multi_table_transaction
4. test_rollback_restores_previous_state

These four tests were specifically designed to validate:

- rollback safety after injection failure
- persistence of committed data across restart
- correctness of multi-table workflow execution
- restoration of the last stable snapshot

This formed the baseline ACID correctness layer.

6.2 Full System Validation Suite

The second experimental layer validated the system under failure-driven and recovery-intensive scenarios.

Experiments performed

The following tests were executed:

- test_failure_after_log_write
- test_failure_before_commit
- test_wal_before_commit_crash
- test_recovery_redo_and_undo
- test_isolation_uncommitted_not_visible
- test_consistency_constraint_violation

These experiments specifically stress-tested:

Failure checkpoints

Simulated crashes were triggered:

- after WAL staged write
- before commit marker
- before commit crash

Recovery paths

The WAL subsystem was tested for:

- committed transaction REDO
- incomplete transaction UNDO
- recovery after malformed or partial staged entries

This ensured that the recovery engine behaves correctly even under abrupt termination.

6.3 Hardening and Reliability Edge-Case Experiments

A third layer of experiments focused on defensive robustness testing.

The following abnormal cases were tested:

- duplicate key insertion
- rollback after failure injection
- restart after repeated operations
- recovery under failure state

These tests verified that:

- no partial duplicate states survive
- repeated operations do not corrupt storage
- rollback remains deterministic
- restart recovery remains stable

This stage was important to validate reliability beyond ideal workloads.

6.4 B+ Tree ACID Validation Experiments

A dedicated B+ Tree validation experiment was executed separately to verify index correctness under transactional semantics.

The experiment included:

1. **Atomicity Test**
   A partial insert was intentionally rolled back.
   Observed result:
   After rollback: [(1, 'A'), (2, 'B')]
   This confirmed:
   no partial keys remain after rollback.
2. **Consistency Test**
   The tree structure was validated for:
   - sorted order
   - no duplicate keys
   - size consistency

   Observed output:
   record_count: 4
   no_duplicates: True
   is_sorted: True
   size_matches: True
3. **Durability Test**
   A persisted key was verified after restart:
   (99, 'Persist')
   This confirmed B+ Tree durability.
4. **Isolation Test**

   The current isolation mode was experimentally verified as:

   Isolation model: auto-commit

   This aligns with the current staged operation design of the engine.

6.5 DB Manager CRUD Workflow Experiment

A final functional experiment validated the application-facing CRUD correctness.

The following workflow was tested:

- insert
- update
- delete
- range query

Observed results:

After insert: [(1, 'Alice'), (2, 'Bob')]

After update: [(1, 'Alicia'), (2, 'Bob')]

After delete: [(1, 'Alicia')]

Range query: [(1, 'Alicia')]

This experiment confirmed that the transaction-safe engine remains functionally correct for normal database usage scenarios.

6.6 Experimental Conclusion

The experimental design successfully validates the system across:

- 17 automated validation tests
- B+ Tree ACID experiments
- CRUD workflow correctness
- rollback and recovery edge cases
- failure injection checkpoints

This layered methodology demonstrates that the implemented Module A engine is robust under correctness validation, injected crashes, restart recovery, and index consistency workloads.

7. Limitations and Future Improvements

While the implemented Module A engine successfully ensures transaction correctness, rollback safety, WAL durability, and B+ Tree consistency, a few practical limitations remain.
The current design follows an *operation-level auto-commit model*, which simplifies rollback and recovery but limits support for long multi-step business transactions and explicit user-controlled transaction sessions. Extending it to support session-based transactions and savepoints would make the system more flexible.
The use of a *single global lock* guarantees correctness and prevents race conditions, but it may reduce performance under high concurrency. Future improvements can introduce finer-grained locking such as row-level or table-level locks, to improve scalability.
Additionally, the *WAL grows continuously* because staged, committed, and rollback entries are logged for every mutation. Although this strengthens recovery reliability, it may increase restart time for larger workloads. Checkpointing and WAL compaction can optimise this in future versions.

Overall, these are design trade-offs made to prioritise correctness and reliability in Module A, while leaving clear scope for future concurrency and performance enhancements.

8. Conclusion

We successfully implemented Module A: Transaction Management, ACID Validation, and Crash Recovery on our custom B+ Tree database system. The engine ensures correct transactions, rollback safety, WAL-based durability, and restart recovery, while maintaining consistency between table records and the B+ Tree index.The successful completion of all 17 validation tests, along with B+ Tree ACID and CRUD demonstrations, confirms that the system remains reliable under both normal operations and injected failures.

Overall, the project provides a robust transactional foundation with clear scope for future improvements in concurrency control and recovery optimisation.

9. Team Members and Contributions

Members Name	Contribution
Nitin Bansal	Module B
Devvrat Hans	Module B
Thipparapu Rushitha	primarily contributed to the transaction workflow design and WAL-based crash recovery implementation, including failure injection checkpoints and restart recovery validation and also actively participated in the video presentation and final report formatting.
Sailaja Mudu	focused on the Database Manager integration, ACID validation test suites, consistency checks, and report preparation, ensuring correctness across rollback, durability, and B+ Tree synchronization and also participated in report writing and video presentation.
Vadithya Harsha Vardhan Nayak	contributed to the B+ Tree transactional consistency, CRUD workflow demonstrations, and reliability edge-case testing, including duplicate handling and restart-based validation scenarios, while also actively participating in the video presentation and preparing the GitHub README/documentation file.